

PDE Solver Project

R

June 8, 2026

Contents

1	Thomas Algorithm	1
2	Implicit Solution to Black-Scholes	3
3	Crank-Nicolson Scheme	7
4	Comparison of Errors	8

Contents

Abstract

This is a high performance Python project to solve partial differential equations (PDEs), specifically the Black Scholes PDE. Towards this end, I implement the Thomas algorithm, using numba and numpy for pre-compiled speed. I also implement the implicit and explicit schemes for solving the Black-Scholes PDE, and finish with an implementation of the joint Crank-Nicolson scheme for a central approximation that achieves lower and more stable root mean squared error than the previous methods.

1 Thomas Algorithm

To support efficient solutions in later questions, we can quickly devise a function implementing the Thomas algorithm to solve a system of equations with a tridiagonal matrix multiplied by a set of solutions that equal known values. Specifically, in the system:

$$A\mathbf{x}_i = \mathbf{d}_i,$$

where:

$$A = \begin{pmatrix} \alpha_1 & \beta_1 & 0 & \dots & 0 \\ \gamma_2 & \alpha_2 & \beta_2 & \ddots & \vdots \\ 0 & \gamma_3 & \alpha_3 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & \beta_{n-1} \\ 0 & \dots & 0 & \gamma_n & \alpha_n \end{pmatrix},$$

the Thomas Algorithm calculates the unknown vector $\mathbf{x}_i = (x_1, x_2, \dots, x_n)^\top$ when we know $\mathbf{d}_i = (b_1, b_2, \dots, b_n)^\top$. This is highly performant, with a time complexity

of $\mathcal{O}(N)$, and only three vectors of length N required in memory. The algorithm functions by converting the tridiagonal matrix into an upper triangular matrix with a normalised diagonal. We can normalise the central diagonal by dividing each i -th line by α_i . We can also turn the matrix into an upper triangular matrix by taking away the respective amount of α_i from the normalised upper line. As a process:

$$A = \begin{pmatrix} 1 & \beta_1/\alpha_1 & 0 & \dots & 0 \\ \gamma_2 & \alpha_2 & \beta_2 & \ddots & \vdots \\ 0 & \gamma_3 & \alpha_3 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & \beta_{n-1} \\ 0 & \dots & 0 & \gamma_n & \alpha_n \end{pmatrix},$$

$$A = \begin{pmatrix} 1 & \beta_1/\alpha_1 & 0 & \dots & 0 \\ 0 & \alpha_2 - \gamma_2(\beta_1/\alpha_1) & \beta_2 & \ddots & \vdots \\ 0 & \gamma_3 & \alpha_3 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & \beta_{n-1} \\ 0 & \dots & 0 & \gamma_n & \alpha_n \end{pmatrix},$$

We can define basic variables:

$$\beta'_i = \begin{cases} \frac{\beta_i}{\alpha_i}, & i = 1, \\ \frac{\beta_i}{\alpha_i - \gamma_i \beta'_{i-1}} & i = 2, 3, \dots, n-1 \end{cases}$$

and,

$$b'_i = \begin{cases} \frac{b_i}{\alpha_i}, & i = 1 \\ \frac{\beta_i - \gamma_i b'_{i-1}}{\alpha_i - \gamma_i \beta'_{i-1}}, & i = 2, 3, \dots, n \end{cases}$$

We can express the problem as:

$$\begin{pmatrix} 1 & \beta'_1 & 0 & \dots & 0 \\ 0 & 1 & \beta'_2 & \ddots & \dots \\ 0 & 0 & 1 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & \beta'_{n-1} \\ 0 & \dots & 0 & 0 & \alpha_n \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_{n-1} \\ x_n \end{pmatrix} = \begin{pmatrix} b'_1 \\ b'_2 \\ \vdots \\ b'_{n-1} \\ b_n \end{pmatrix}$$

Which is elementary to evaluate, since we can calculate each individual x_i as the only unknown in the equation. This saves considerable time over other matrix solving algorithms like Gaussian elimination as the solution to each individual equation is already given and can be given in terms of the original x_i mapping. As a program, we can describe the process as:

```
def tridiagonal_solve_thomas (alpha, beta, gamma, b):

    initialise hat_alpha[0] to alpha[0]
    initialise hat_b[0] to b[0]

    # calculate the transformed A matrix with a forwards loop
    for i between 2 and n+1:
        hat_alpha[i] = alpha[i] - ((beta[i-1] * gamma[i-1]) /
            hat_alpha[i-1])
        hat_b[i] = b[i] - ((hat_b[i-1] * gamma[i-1]) /
            hat_alpha[i-1])
```

```

# complete the simple calculation for x_i
x = hat_b / hat_alpha

for i between n-2 and -1:
    x[i] = (hat_b[i] - beta[i] * x[i+1]) / hat_alpha[i]

return x

```

This can be implemented relatively easily, and, against a matrix we can solve, it produces accurate results. For example:

$$A = \begin{pmatrix} 3 & 6 & 0 & 0 \\ 9 & 4 & 2 & 0 \\ 0 & 7 & 10 & 8 \\ 0 & 0 & 12 & 4 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} 10 \\ 6 \\ 2 \\ 4 \end{pmatrix},$$

this algorithm provides the true value of x : `array([-0.49084249, 1.91208791, 1.38461538, -3.15384615])`. We can test this more robustly against the `scipy` bounded system of equations solver. We can use a simple assertion block and generate a list of random testing values to test whether `tridiagonal_solve_thomas` solves problems exactly.

```

# isclose can account for floating point imprecision (1e-15 for
# np.float64)
try:
    assert np.isclose(thomas_solutions[i], scipy_solutions[i],
                      rtol = 1e-12, atol = 1e-15)
except AssertionError:
    print(f"Error on trial {size}.")
    return False

```

Over a set of 9 trials across sample sizes, including a $10,000 \times 10,000$ dimensional matrix problem, this function passes all tests, suggesting that it is effectively implemented. For efficiency, the biggest hurdle was naturally slow Python loop logic. Luckily, the Python `numba` package compiles functions rather than using the slower Just in Time (JIT) logic by the Python compiler, allowing near C-runtime speeds. This integrates with `numpy` to ensure speed and accuracy. Looking at the time efficiency of both on a logged plot, Figure 1, we can see that while the `scipy` algorithm is roughly as asymptotically efficient, this Thomas algorithm has a slightly lower constant multiplier. It is clear looking at the reference lines, that both are $\mathcal{O}(n)$. The `scipy` solver is optimised for diagonal matrices in general, rather than just tridiagonal ones, consequently the Thomas algorithm is faster in the long run. Both are precompiled, it is possible that these aggressive optimisations are behind the lower runtime than $\mathcal{O}(N^2)$, while the Thomas algorithm is consistently more efficient, the `scipy` algorithm becomes faster for matrices greater than 1000×1000 .

2 Implicit Solution to Black-Scholes

Suppose we want to solve the Black-Scholes PDE:

$$\frac{1}{2}\sigma^2 s^2 \frac{\delta^2 V(s,t)}{\delta s^2} + rs \frac{\delta V(s,t)}{\delta s} - rV(s,t) + \frac{\delta V}{\delta t} = 0$$

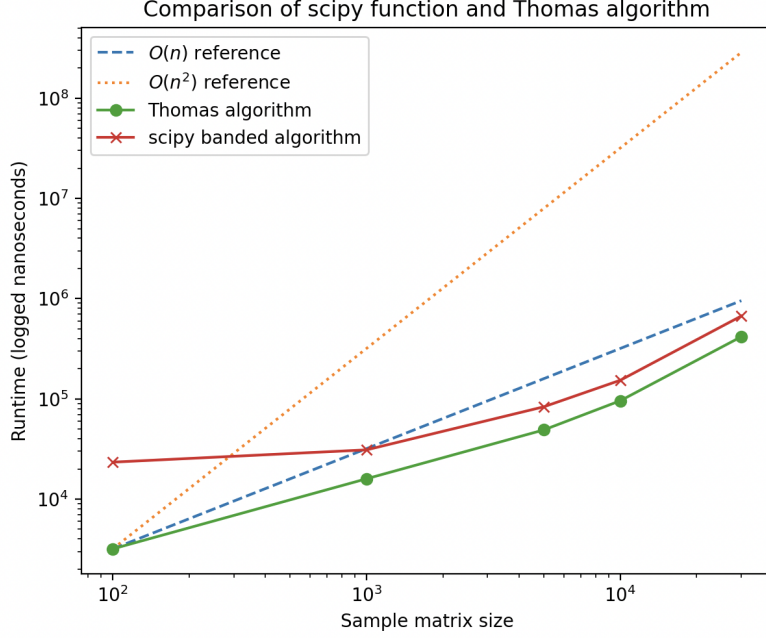


Figure 1: Comparison of Thomas Algorithm and scipy solver (logged plot)

Where r is the interest rate, σ is the volatility, and S_t is the value of the stock. This is true when dS_t a martingale under $\mathbb{Q}$, or when $\mu = r$. Untransformed, this has the boundary conditions:

$$\lim_{s \rightarrow 0} V(s, t) = r_t(t), \forall t \in [0, T]$$

$$\lim_{s \rightarrow \infty} \frac{V(s, t)}{r_2(s, t)} = 1, \forall t \in [0, T]$$

$$V(s, T) = \max(0, S_T - K)$$

where K is the strike price.

However, we can convert the Black-Scholes PDE into the classical heat equations using a change of variables. This is given:

$$\frac{\partial y}{\partial \tau} = \frac{\partial^2 y}{\partial x^2}, \quad (1)$$

where the change of variables are:

$$x = \log(s) + \left(r - \frac{1}{2}\sigma^2\right)(T - t)$$

$$\tau = \frac{\sigma^2}{2}(T - t)$$

$$y(x, \tau) = \exp\left(-r\left(T - \frac{2\tau}{\sigma^2}\right)\right) V\left(\exp\left(x - \left(\frac{2r}{\sigma^2} - 1\right)\tau\right), T - \frac{2\tau}{\sigma^2}\right),$$

this instead has the boundary conditions:

$$\begin{aligned}\lim_{x \rightarrow -\infty} y(x, \tau) &= e^{-r\left(T - \frac{2\tau}{\sigma^2}\right)} r_1 \left(T - \frac{2\tau}{\sigma^2}\right) \\ \lim_{x \rightarrow \infty} \frac{y(x, \tau)}{e^{-r\left(T - \frac{2\tau}{\sigma^2}\right)} r_2 \left(e^{x - \left(\frac{2r}{\sigma^2} - 1\right)}, T - \frac{2\tau}{\sigma^2}\right)} &= 1 \\ y(x, 0) &= e^{-rT} h(e^x)\end{aligned}$$

The implicit scheme can solve this without any condition on the size of M , permitting highly efficient solutions. The implicit scheme hinges on central differencing approximations to equation (1), where:

$$\frac{w_{i,v} - w_{i,v-1}}{\delta\tau} - \frac{w_{i+1,v} - 2w_{i,v} + w_{i-1,v}}{(\delta x)^2} = 0$$

Becomes the approximation to (1). After some algebraic work, it can be shown that:

$$\mathbf{A}_{im} \mathbf{w}_{v+1} = \mathbf{w}_v + \mathbf{d}_v$$

where,

$$\mathbf{A}_{im} = \begin{pmatrix} 1+2\lambda & -\lambda & 0 & \dots & 0 \\ -\lambda & 1+2\lambda & -\lambda & \dots & \vdots \\ 0 & -\lambda & 1+2\lambda & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & -\lambda \\ 0 & \dots & 0 & -\lambda & 1+2\lambda \end{pmatrix},$$

$$\lambda = \frac{\delta\tau}{(\delta x)^2},$$

$$\mathbf{d}_v = (\lambda w_{0,v+1}, 0, \dots, 0, \lambda w_{M,v+1}),$$

and where w is the discounted stock price evolutionary process of note. This is conveniently a system of equations that can be solved by the performant Thomas algorithm, allowing us to use the earlier function.

```
def black_scholes_implicit (r, sigma, T, K, M, N, xmin, xmax):

    # Approximation constants
    initialise delta_tau as ((sigma^2) * T) / (2 * N)
    initialise delta_x as (xmax - xmin) / M
    initialise lambda = delta_tau / (delta_x ** 2)

    # Approximation evolution
    Declare w as a M+1 by N+1 matrix

    # initial period value
    for i between 0 and M+1:
        set the current x
        set w[i][0] as the payoff for that x
```

```

# central differenced matrix A
instantiate alpha, beta, and gamma as vectors to save
memory

for v between 0 and N:

    # boundary conditions
    set the left boundary w[0, v+1]
    set the right boundary w[M, v+1]

    # w_v vector (copy of last chain)
    set RHS as a copy of the v chain

    # addition of boundary conditions (RHS is now w_v + d_v
    )
    add lambda * w[0, v+1] to the first entry of RHS
    add lambda * w[M, v+1] to the last entry of RHS

    # Solve for current period step (since Aw_{v+1} = (w_v
    + d_v))
    calculate the next period step using
    tridiagonal_solve_thomas

return w_{i,v} for i in M and v in N

```

We can run this function to see the values it yields. Observing the first run (Figure 2), we can see that the value is not initially convergent on the left bound, but, by the final run, the approximation has met both bounds and very closely approximates the exact option price (given by the Black-Scholes equation) across the range of possible stock prices. The exact values are:

The final exact value is £2076.92, the initial implicit estimate is £2019.00, and the final implicit estimate is £2076.92.

Listing 1: Initial and final approximated option price values at boundaries

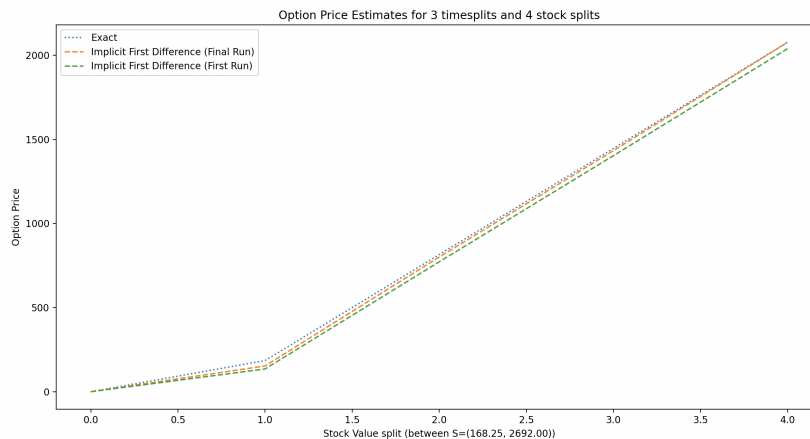


Figure 2: Implicit Method Option Price Approximation

A number of optimisations are made internally. The tridiagonal matrices are never handled as actual matrices, but vectors for each diagonal. All values are set to `np.float64`'s to ensure numba works consistently. The actual function is also wrapped so that input validation logic can be applied. We can efficiently apply all the boundary conditions across every value of $w_{i,v}$ using numpy, rather than looping over the values. The diagonals of A_{im} can likewise be calculated as a function of λ multiplying across the stock price splits.

3 Crank-Nicolson Scheme

The Crank-Nicolson Scheme is a variation on the implicit and explicit method that adds together both equations to acquire the relation:

$$A_{CN}\mathbf{w}_{v+1} = B_{CN}\mathbf{w}_v + \mathbf{d}_{CN,v},$$

where,

$$A_{CN} = A_{im} + I = \begin{pmatrix} 2+2\lambda & -\lambda & 0 & \dots & 0 \\ -\lambda & 2+2\lambda & -\lambda & \dots & \vdots \\ 0 & -\lambda & 2+2\lambda & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & -\lambda \\ 0 & \dots & 0 & -\lambda & 2+2\lambda \end{pmatrix},$$

(where I is the identity matrix) and,

$$B_{CN} = A_{ex} + I = \begin{pmatrix} 2-2\lambda & \lambda & 0 & \dots & 0 \\ \lambda & 2-2\lambda & \lambda & \dots & \vdots \\ 0 & \lambda & 2-2\lambda & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & \lambda \\ 0 & \dots & 0 & \lambda & 2-2\lambda \end{pmatrix},$$

and,

$$\mathbf{d}_v = (\lambda(w_{0,v} + w_{0,v+1}), 0, \dots, 0, \lambda(w_{M,v} + w_{M,v+1})),$$

This has desirable error properties, and is far more numerically stable than the implicit method alone. The program is largely the same as the previous scheme, except we also calculate $B_{CN}\mathbf{w}_v$ using the Stencil scheme for multiplying diagonal matrices. This is a highly optimised algorithm which calculates the product as the dot product of 3 vectors of length M , which numba optimises extremely well, we also cache diagonal values between runs. Additionally, we change the boundary conditions to reflect the new $\mathbf{d}_i$. Observing Figure 3, both approximations provide extremely similar results, and the same values at the boundaries (Listing 2). We can see, however, that the first Crank-Nicolson approximation is far closer than the first implicit approximation, we can dive deeper into the numerical stability of these two models.

```
The final exact value is £2076.92, the initial Crank Nicolson
estimate is £2019.00, and the final Crank Nicolson estimate
is £2076.92.
```

Listing 2: Initial and final approximated option price values at boundaries (Crank-Nicolson)

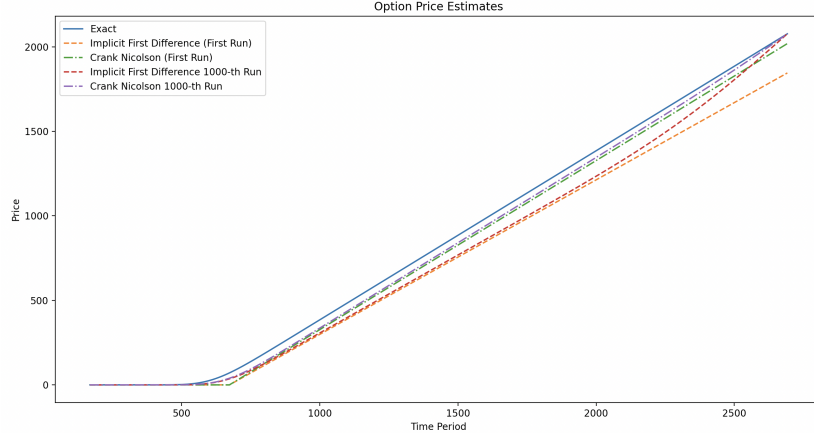


Figure 3: Convergence across Models

4 Comparison of Errors

We know from the established literature that the implicit scheme has errors of order $\mathcal{O}(\delta\tau) + \mathcal{O}(\delta x)^2$, while the Crank-Nicolson scheme has errors of order $\mathcal{O}(\delta\tau)^2 + \mathcal{O}(\delta x)^2$. We can observe this looking at the previous figures, particularly 3, where the Crank-Nicolson scheme is both closer with its initial estimate and generally better fitting than the implicit scheme alone.

Heatmaps were also generated for the Root Mean Squared Error (RMSE) across both schemes. We can observe in Figure 5, that there is very little observable gradation in RMSE along values of N , implying that convergence in time space is fast. Comparing this to 4, we can observe clear "squaring" where obvious jumps in RMSE quality occur across different values of N (this is most clear between around $N \approx 190$), implying a far slower convergence. The numerical stability engendered by lower RMSE ensures much quicker convergence on the exact price than the implicit scheme, enabling potentially faster and lower variance estimates despite the increased computational complexity from calculations with the A_{ex} vector. It is clear from the empirical error values that the implicit scheme has slower dt convergence than the Crank-Nicolson scheme, but dx is still a dominating term in convergence.

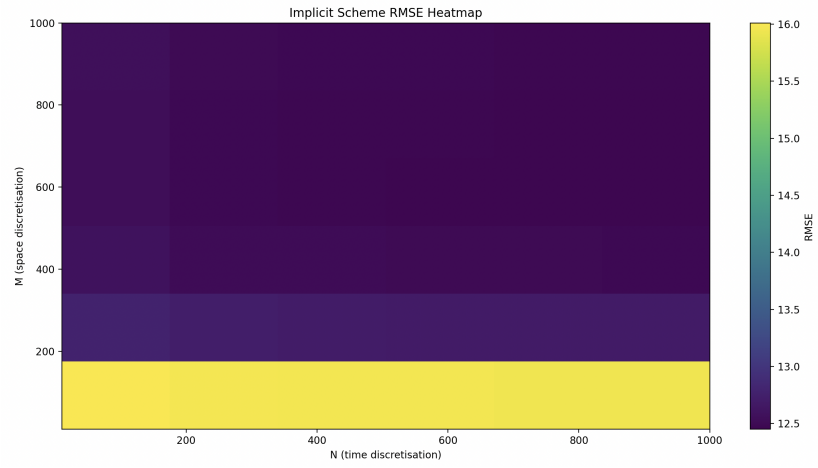


Figure 4: Implicit Scheme Root Mean Squared Error (RMSE) across N and M

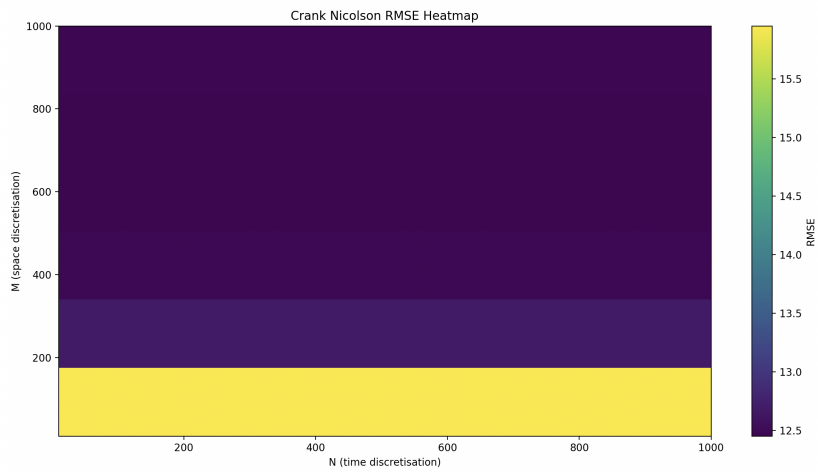


Figure 5: Crank-Nicolson Scheme Root Mean Squared Error (RMSE) across N and M